# Structuring the empirical chapters

A skeleton for the data-collection / results and discussion / evaluation sections, with the figure and number for each claim already attached. Figure numbers refer to `docs/report_figures/`.

The two chapters can be combined — the structure below works either as two chapters or as one chapter with results and interpretation interleaved per axis. **If you combine them, do it per axis** (tolerance: result then interpretation; cost: result then interpretation), not by putting all results first and all discussion second. Mixed-order combining is where these chapters usually become hard to follow.

---

## Chapter A — Experimental method and data collection

The job of this chapter is to make the results *believable before they are read*. Everything here is about controls.

### A.1 What is held constant, and why it has to be

The central methodological claim: comparing erasure codes is meaningless unless storage overhead is fixed, because any code tolerates more erasure if allowed more space. Every scheme runs at **$n \approx 8192$, $k = 4096$, ~2× overhead** (2D-RS at 2.06×, within 3%).

State plainly that an earlier iteration had 2D-RS at 4× overhead, and that comparing a 4×-overhead code against 2×-overhead codes at equal erasure rate flattered it. Describing a correction you made is a strength, not an admission — it shows the control was deliberate.

### A.2 The fairness controls

Four, in increasing order of strictness:

1. **One field.** Real BLS12-381 scalar field, not a toy field.
2. **One blob.** Real application bytes packed once via EIP-4844 rules; every scheme encodes that same blob, not seven random ones.
3. **Identical erasure positions.** Where codeword lengths match — 1D-RS and all five BC rows are $n = 8192$ — trials corrupt the *same positions*, not merely the same rate. 2D-RS has $n = 8448$ and so draws its own masks at the same rate. Say this explicitly; it is the strictest control available and a reader will wonder.
4. **Success is measured in bytes.** A trial counts only if the recovery round-trips to the exact original application bytes.

### A.3 Why byte-level verification, not the decoder's own flag

This deserves its own subsection, and **figure 13**. An earlier decoder returned zeros for values it had not solved and reported success: across 40 trials it claimed success every time and was wrong on 31. Had the harness trusted the `remaining` flag, it would have reported 100% success at every erasure rate.

This is the strongest methodological point in the project. It shows the validation caught a fault that would have invalidated every number, and it justifies the expense of byte-level checking.

### A.4 Trial allocation and the threshold definition

- Trial counts vary by scheme and rate — more near a transition where outcomes genuinely vary, fewer where they do not. Give the rule, and note that 95% Wilson intervals are in the JSON.
- **Define the threshold explicitly**: the erasure rate at which the success rate crosses 50%, linearly interpolated between the two swept rates that bracket it. Every tolerance number in the report depends on this definition, so it must be stated once, early, and not changed.

### A.5 Reproducibility

Measurement writes JSON; every figure and table is computed from that JSON rather than transcribed. Figure generation is deterministic. State the pipeline — `benchmark_1D_2D_BC.py` → `benchmark_results.json` → `make_figures.py` — and that 520 trials underlie the sweep.

---

## Criteria coverage — check this before you write

The research question names six axes, and the Phase 3 brief names three things the network layer must capture. Which figure answers which:

| criterion | coding benchmark | network simulation |
|---|---|---|
| Fault tolerance | `01`, `02`, `03` | `01`, `02` |
| Computational complexity | `04`, `05` | `07` |
| Encoding / decoding efficiency | `04`, `06` | `06`, `07` |
| Storage overhead | held fixed at ~2× — stated in method, not a figure | — |
| Bandwidth consumption | `08` (data needed) | `04` (data fetched) |
| Scalability | — | `08` (peer count) |

| Phase 3 brief asks for | covered by |
|---|---|
| Bandwidth saturation | network `04`, at 25 Mbps with six blobs |
| Packet propagation latency | network `03` (gather time against withholding) |
| Block processing throughput | network `05` (blobs per slot), `06` (fits in a slot) |
| Varying adversarial conditions | every network figure sweeps it |
| **Standard network architectures (plural)** | **not covered — one topology only** |

Two things worth noticing in that table. **Storage overhead has no figure** because it is a controlled variable, not a result — say so explicitly in §A.1 rather than leaving a reader to wonder where it went. And **bandwidth is measured twice by independent means**: the benchmark computes what a scheme needs from its parameters, the simulation measures what a node actually pulled. They agree on the ordering, which is worth one sentence in §C — two methods agreeing is evidence, not repetition.

---

## Chapter B — Results

One section per axis from the research question. Report numbers; save interpretation for Chapter C (or the paired subsection, if combining).

### B.1 Fault tolerance — figures 1, 2

| scheme | measured threshold |
|---|---|
| 1D-RS | 0.505 |
| 2D-RS | 0.397 |
| BC ω512 λ2 | 0.350 |
| BC ω1024 λ2 | 0.341 |
| BC ω2048 λ2 | 0.340 |
| BC ω1024 λ3 | 0.275 |
| BC ω1024 λ4 | 0.210 |

Two findings:

- **1D-RS reaches 0.505 against a theoretical maximum of 0.50** — what being MDS means, and a sanity check that the harness is sound.
- **The block-circulant threshold is set by λ alone.** The three λ=2 rows land at 0.340, 0.341 and 0.350 while μ ranges over 2, 4, 8 and ω over 512 to 2048. Against the $1/(1+\lambda)$ prediction of 0.333, 0.250, 0.200, the measurements are 0.34, 0.28, 0.21. Figure 2 is theory and experiment agreeing — lead with it.

### B.2 Computational cost — figures 4, 6

Report encode and decode separately; they are not interchangeable. Decode at 20% erasure: 1D-RS 20.4 s, 2D-RS 2.7 s, BC rows 1.5–5.5 s. Encode runs the other way: 1D-RS 0.23 s, BC ω2048 15.7 s.

Note that all timings are single-threaded Python over `galois`, so they are **relative** measurements, not production figures. Say so once here rather than defending it later.

### B.3 Bandwidth — figure 8

Surviving data needed to rebuild one blob, as a multiple of blob size: 1D-RS 0.99× (the floor), everything else 1.24–1.58×.

### B.4 Network behaviour — network figures 01–08

The simulation's distinguishing feature: in the benchmark the erasure mask is an *input you invent*; here it is an *output* — whatever failed to arrive before the deadline. Make that the opening sentence of the section.

- Realised erasure ≈ $p^4$ under custody replication 4 (network `01`), identical across all six schemes.
- Schemes fail in the same order as the coding benchmark predicted (network `02`), via an entirely different mechanism. **This is the most valuable result in the project** — two independent methods agreeing is far stronger evidence than either alone. Do not bury it.

- **Latency** (network `03`): withholding costs time, not just data, because the client must burn its request timeout before falling back. The scheme needing the most cells hits the deadline first — BC ω1024 λ4 at 50% withholding, against 80% for 1D-RS.
- **Bandwidth** (network `04`): 1.04 MiB fetched by 1D-RS against 1.54 MiB by BC ω1024 λ4, for 0.73 MiB of payload. Same ordering as the benchmark's surviving-data figure, arrived at independently.
- **Throughput** (network `05`): blobs reconstructed per slot inverts the tolerance ordering — the scheme surviving the most erasure rebuilds the fewest blobs.
- **Does it fit in a slot?** (network `06`): at six blobs and no adversaries only BC ω1024 λ3 and λ4 finish inside twelve seconds; past about 20% withholding nothing does. 1D-RS takes 354 s — roughly thirty slots of work for one slot of data.

### B.5 Scalability — **figure 12**

Realised erasure against peer count, against the exact hypergeometric probability computed from the model rather than fitted. Measurement lies on prediction. Conclusion: what protects a cell is the *fraction* of the network custodying it, so peer count cannot be varied without specifying replication.

---

## Chapter C — Discussion and evaluation

### C.1 Guarantees versus behaviour — **figure 3**

2D-RS tolerates 0.397 against a worst-case guarantee of 0.098 — four times its own bound. Not a contradiction: a minimum-distance guarantee holds against *every* pattern including the adversarial worst case, while these trials draw patterns at random.

This distinction is worth a paragraph of its own, because it is the most likely place for a reader to think they have found an error.

### C.2 Why the costs differ — **figure 5**

Raw decode time rises with erasure rate for everything, which mostly says there is more to repair. Divide it out and the structure appears: 1D-RS flat at 10–13 ms per erasure (cost linear in erasures, one long division's worth each); BC rows at 1–3.5 ms; 2D-RS falling from 8 ms to 1 ms, revealing a large fixed cost that amortises — which is why it looks expensive at 2% erasure and cheap at 30%.

### C.3 The trade-off — **figure 7**

The synthesis. Only three of seven schemes are on the cost-versus-tolerance frontier. BC ω512 λ2 is the cheapest scheme still surviving a third of the codeword; 2D-RS buys ∼5 points more tolerance for roughly double the decode; 1D-RS buys the maximum and pays an order of magnitude.

Call out explicitly that **BC ω1024 λ4 is dominated on both axes** by BC ω512 λ2 — a parameter choice with nothing to recommend it. Negative results about your own parameter space are credible and worth stating.

### C.4 The asymmetry that changes the recommendation — **figure 6**

Encode is paid once by the proposer; decode is paid by every reconstructing node. 1D-RS encodes in 0.23 s and decodes in 20 s; BC ω2048 spends 15.7 s encoding to decode in 5.5 s. For data-availability sampling

the second shape is the more useful one — an argument for block-circulant the tolerance numbers alone do not make.

### C.5 The result that reframes the rest — figure 11

Every scheme gathers in about a second; decode spans three orders of magnitude above that and misses the 4-second sampling deadline. **Bandwidth was the expected constraint and is not close.** Decode arithmetic is the binding one.

This has a consequence worth stating: it reorders the research question's own priorities. A scheme's erasure tolerance matters less than expected, because custody replication absorbs most peer failure (§B.4) and decode speed dominates the time budget.

### C.6 Threats to validity

Be first to raise these. A reader who finds them unprompted discounts everything else.

- **Random, not adversarial, erasures.** Patterns are drawn i.i.d.; a real adversary chooses the worst pattern. Measured tolerances are therefore upper bounds on adversarial performance — directly relevant to 2D-RS and the $\lambda=2$ rows, which exceed their worst-case guarantees.
- **Single-threaded Python.** Relative comparisons hold; absolute seconds would not survive an optimised implementation.
- **The block-circulant rows carry a ~2.3× implementation penalty** that the Reed-Solomon rows do not. Defining the BC local code over roots of unity (to make it FFT-capable) means solving over 255-bit field elements instead of small integers, and that cost is paid even with the transform switched off. Measured in `decoder_version_results.json`. This narrows the apparent 1D-RS gap and moves the 2D-RS crossover; it does not touch any tolerance result. Raise it yourself in §B.2 — it is the kind of thing a reader finds and then distrusts everything else.
- **One blob of application data.** Erasure patterns are reseeded per trial, but the payload is fixed.
- **Block-circulant is measured single-phase.** Phase 2 does not implement the paper's auxiliary construction, so BC thresholds are what phase 1 alone achieves — a limit of this implementation, not of the code. Say which.
- **One network topology**, where the brief asks for several.
- **Simulated network.** Latency and bandwidth are modelled, not measured on real hardware.

### C.7 Limitations and further work

Three concrete items, each with a named fix:

1. Implement Algorithm 1's auxiliary GRS construction for phase 2 — would raise the correction radius from $\rho$ toward $2\rho$ and drop the cost from cubic to quadratic.
2. Replace the NTT path's polynomial division with Newton iteration — the transform is only 12% of that decode; ~80% is a schoolbook division.
3. Add further network topologies.

---

## Two things to settle before writing

**The NTT decoder's status — settled.** The roots-of-unity version stays; nothing is being reverted and no re-run is needed. The decode numbers in the report are the ones to write about. The ~2.3× penalty this carries for the block-circulant rows is measured, stated in the report, and listed under threats to validity above — handle it there rather than by adjusting numbers.

**Where the NTT investigation lives.** It is a genuine negative result — implemented, measured, 7–23× slower, with the cause identified — but it is about *implementation*, not about the coding schemes. It belongs in an implementation appendix or a short subsection of C.7, not in the empirical results. Its four figures are in `docs/figures/` and deliberately left out of `docs/report_figures/`.

---

## Figure budget

Thirteen figures is a lot for one chapter. If you need to cut, in order of what goes first:

| keep | if space |
|---|---|
| 07 (the trade-off — if you keep one, keep this) | 03 |
| 01, 02 (tolerance, and theory agreeing) | 05 |
| 11 (the bottleneck) | 08 |
| 10 (the two methods agreeing) | 12 |
| 04 (decode cost) | 13 (move to methodology) |
| 09 ($p^4$) | 06 |